Ipopt revisited: Strategies to solve edge cases

Horacio Lisdero Scaffino

May 21, 2026

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

Agenda

- A tour of Ipopt
  - History
  - Techniques applied
- An attempt of a simple IPM algorithm
  - Challenges encountered
  - Results
- A tour of ripopt
- Experiments
  - What can we learn
- Conclusion

Ipopt: Interior Point OPTimizer

Popular open-source non-linear optimization package

Bindings available for:

- AMPL (algebraic modeling language)
- Fortran
- GAMS (general algebraic modeling system)
- Julia
- Matlab / Scilab
- .NET languages: C#, F# and Visual Basic.NET
- Python (several wrappers available)
- R
- Rust

Ipopt: The origins

- PhD thesis by Andreas Wächter in 2002 at Carnegie Mellon [1]
- Since 2002 part of the COIN-OP foundation [2]
- Original implementation in Fortran 77 (1.x and 2.x), now deprecated
- Open source since the start
- Benchmarked against KNITRO (1999) and LOQO (1999) [3, 4]

Ipopt: Subsequent paper

- Paper by Wächter and Biegler in 2006 [5]
- More focused on the central algorithm of Ipopt
- Features described more in detail
  - Feasibility restoration phase,
  - Inertia correction,
  - Various heuristics...
- More benchmarks: CUTEr test set [6]

Ipopt: C++ version

- IBM Research decided to invest in an open source re-write of Ipopt in C++
- With the help of Carl Laird, the code was re-implemented from scratch [7]
- 3.x published in Aug 2005
- It is still actively developed on GitHub

Ipopt: Summary of applied techniques

- Primal-dual interior-point method with filter line-search globalization
- Sparse symmetric KKT solves with inertia monitoring/correction
- Inertia correction and regularization to obtain the proper search direction
- Feasibility restoration phase for difficult or infeasible iterates
- Second-order correction steps to reduce Maratos-type effects
- Practical heuristics for robustness and faster performance

Interior Point Methods: Intro

$$\min_{x \in \mathbb{R}^n} \quad \varphi_\mu(x) := f(x) - \mu \sum_{i=1}^{n} \ln\left(x^{(i)}\right) \tag{1}$$

$$\text{s.t.} \quad c(x) = 0 \tag{2}$$

$$x \geq 0 \tag{3}$$

We recover the Karush-Kuhn-Tucker (KKT) conditions for $\mu = 0$:

$$\nabla f(x) + \nabla c(x)\lambda - z = 0 \tag{4}$$

$$c(x) = 0 \tag{5}$$

$$XZe - \mu e = 0 \tag{6}$$

$$x, z \geq 0, \qquad \lambda \in \mathbb{R}^m, \qquad z \in \mathbb{R}^n,$$

$$Z := \operatorname{diag}(z), e = (1, \ldots, 1)^T$$

n: number of primal variables, m: number of equality constraints

KKT system: Derivation

$$F_\mu(x, \lambda, z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

Iteratively solve using Netwon's method:

- Fix a starting point (x_k, λ_k, z_k),
- Calculate the deltas for each dimension $(d_k^x, d_k^\lambda, d_k^z)$
- Use the deltas to move to $(x_{k+1}, \lambda_{k+1}, z_{k+1})$ (more later)

$$F_{\mu_j}(x_k, \lambda_k, z_k) + \nabla F_{\mu_j}(x_k, \lambda_k, z_k)(d_k^x, d_k^\lambda, d_k^z)^T = 0$$

The term $\nabla F_\mu(x, \lambda, z)$ is a 3x3 matrix.
It is the Jacobian of the vector field.
We can move $F_\mu(x, \lambda, z)$ to the other side to get the form $Ax = b$.

KKT system: Matrix form

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \\ d_k^z \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}$$

$$\begin{aligned} \text{where} \qquad A_k &:= \nabla c(x_k), \\ W_k &:= \nabla^2_{xx} \mathcal{L}(x_k, \lambda_k, z_k), \\ \mathcal{L}(x, \lambda, z) &:= f(x) + c(x)^T \lambda - z. \end{aligned}$$

But this system can be further simplified!
Multiply out the third equation:

$$\begin{aligned} Z_k d_k^x + X_k d_k^z &= \mu_j e - X_k Z_k e \\ d_k^z = \mu_j X_k^{-1} e - z_k - X_k^{-1} Z_k d_k^x \\ \Sigma_k &:= X_k^{-1} Z_k \end{aligned}$$

KKT system: Symmetric matrix form

Now d_k^z only appears in the first equation as $-Id_k^z = -d_k^z$ which we move to the other side:

$$
\begin{aligned}
W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + d_k^z \\
W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + \\
&\qquad + \mu_j X_k^{-1} e - z_k - \Sigma_k d_k^x \\
(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - \mu_j X_k^{-1} e) \\
(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k)
\end{aligned}
$$

Reconstructing the system, we get:

$$
\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix} \tag{7}
$$

KKT system: Important considerations

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

- Symmetric system
- Matrix in top-left block projected onto the null space of the constraint Jacobian A_k^T is positive definite
- If A_k does not have full rank, the iteration matrix is singular and a solution may not exist

Solve instead the following system with **inertia correction** for some $\delta_w, \delta_c \geq 0$:

$$\begin{bmatrix} W_k + \Sigma_k + \delta_w I & A_k \\ A_k^T & -\delta_c I \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

I wanted to contribute something back to the community,

i.e, **not** throw away the Python code 1 week after the seminar

The original plan

1. Select the most popular optimization package
2. Implement a simple interior-point method
3. Compare it with Ipopt
4. Profit!

argmin

argmin v0.11.0

Numerical optimization in pure Rust

Homepage Documentation Repository

All-Time: 3,049,452

Recent: 531,942

Updated: 7 months ago

Website [8]

Pure Rust 🦀

We aim at providing a wide range of solvers implemented in pure Rust.

Consistent interface 🔌

Easily plug your optimization problem in different solvers thanks to a consistent interface.

Type agnostic 🔧

Represent your variables, gradients, Jacobians and Hessians using your own types.

Various math backends 🧮

Use Vecs, ndarray, or nalgebra for behind-the-scenes math. Or implement your own backend.

Observers 👀

Observe the progress of your optimization by logging to screen or file or implement your own observer.

Checkpointing 🚩

Mitigate the negative effects of crashes in unstable computing environments by saving regular checkpoints.

Extensible 🪡

Most features can be exchanged for user supplied implementations thanks to Rusts powerful generics and traits.

Development framework 🏗️

Use argmins tools and ecosystem to implement your own solver – with all the features mentioned above.

discarded several libraries: dealt only with LP (`good_lp`), used novel algorithms (`clarabel`), or were too small (`interiors`)

argmin: Code example

```rust
impl<O, P, G, H, F> Solver<O, IterState<P, G, (), H, (), F>> for Newton<F>
where
    O: Gradient<Param = P, Gradient = G> + Hessian<Param = P, Hessian = H>,
    P: Clone + ArgminScaledSub<P, F, P>,
    H: ArgminInv<H> + ArgminDot<G, P>,
    F: ArgminFloat,
{
    fn next_iter(
        &mut self,
        problem: &mut Problem<O>,
        mut state: IterState<P, G, (), H, (), F>,
    ) -> Result<(IterState<P, G, (), H, (), F>, Option<KV>), Error> {
        let param = state.take_param().ok_or_else(argmin_error_closure!(
            NotInitialized,
            concat!(
                "`Newton` requires an initial parameter vector. ",
                "Please provide an initial guess via `Executor`s `configure` method."
            )
        ))?;
        let grad = problem.gradient(&param)?;
        let hessian = problem.hessian(&param)?;
        let new_param = param.scaled_sub(&self.gamma, &hessian.inv()?.dot(&grad));
        Ok((state.param(new_param), None))
    }
}
```

Math backends

The math backend is implemented in a separate crate `argmin-math`

It is trait-based: interfaces define the methods required from the linear algebra backend

- `nalgebra` [9]
- `faer` [10]

Add a solver

Add a trait:

```rust
pub trait ArgminLuSolve<T> {
    /// Solve linear system using LU decomposition.
    fn lu_solve(&self, rhs: &T) -> Result<T, Error>;
}
```

And the corresponding implementation:

```rust
impl<N, D, C, S> ArgminLuSolve<OMatrix<N, D, C>> for SquareMatrix<N, D, S>
where
    N: ComplexField,
    D: Dim + DimMin<D, Output = D>,
    C: Dim,
    S: Storage<N, D, D>,
    DefaultAllocator: Allocator<N, D, D> + Allocator<N, D, C> + Allocator<N, D>,
{
    #[inline]
    fn lu_solve(&self, rhs: &OMatrix<N, D, C>) -> Result<OMatrix<N, D, C>, Error> {
        let lu = LU::new(self.clone_owned());
        lu.solve(rhs).ok_or_else(|| LuSolveError {}.into())
    }
}
```

No constraints?

```rust
/// Defines equality constraints for a problem.
///
/// It follows the form h(x) = (0, ... , 0), where h is a vector field.
pub trait EqualityConstraint {
    /// Type of the parameter vector
    type Param;
    /// Type of the return value of the equality constraints
    type Output;

    /// Compute all equality constraints
    fn equality_constraint(&self, param: &Self::Param) -> Result<Self::Output, Error>;
}
```

And their derivatives...

```rust
/// Defines the Jacobian of the vector field representing the equality constraints for a problem.
pub trait EqualityConstraintJacobian {
    /// Type of the parameter vector
    type Param;
    /// Type of the Jacobian
    type Jacobian;

    /// Compute the Jacobian of the equality constraints
    fn equality_constraint_jacobian(&self, param: &Self::Param) -> Result<Self::Jacobian, Error>;

    bulk!(equality_constraint_jacobian, Self::Param, Self::Jacobian);
}
```

And the Hessian of the Lagrangian...

```rust
pub trait LagrangianHessian {
    /// Type of the parameter vector
    type Param;
    /// Type of the multipliers associated with equality constraints
    type MultipliersEq;
    /// Type of the multipliers associated with inequality constraints
    type MultipliersIneq;
    /// Type of the Hessian
    type Hessian;

    /// Compute the Hessian of the Lagrangian
    fn lagrangian_hessian(
        &self,
        param: &Self::Param,
        multipliers_eq: &Self::MultipliersEq,
        multipliers_ineq: &Self::MultipliersIneq,
    ) -> Result<Self::Hessian, Error>;

    bulk!(
        lagrangian_hessian,
        Self::Param,
        Self::MultipliersEq,
        Self::MultipliersIneq,
        Self::Hessian
    );
}
```

And iteration state for NLP...

- `param: Option<P>`
- `slacks: Option<S>`
- `best_param: Option<P>`
- `best_slacks: Option<S>`
- `cost: F`
- `best_cost: F`
- `target_cost: F`
- `grad: Option<G>`
- `lagrangian_hessian: Option<LH>`
- `equality_constraints: Option<EqC>`
- `inequality_constraints: Option<IneqC>`
- `equality_constraint_jacobian: Option<EqCJ>`
- `inequality_constraint_jacobian: Option<IneqCJ>`
- `mu: Option<F>`
- `lambda_eq: Option<LambdaEq>`
- `lambda_ineq: Option<LambdaIneq>`
- `alpha_primal: Option<F>`
- `alpha_dual: Option<F>`
- `inf_pr: Option<F>`
- `inf_du: Option<F>`
- `compl_inf: Option<F>`
- `iter: u64`
- `last_best_iter: u64`
- `max_iters: u64`
- `counts: HashMap<String, u64>`
- `counting_enabled: bool`
- `time: Option<Duration>`
- `termination_status: TerminationStatus`

Finally! IPM

`nalgebra_backend.rs`: All the linear-algebra-specific logic for nalgebra, the only supported backend

`linesearch.rs`: The custom backtracking linear search. The existing implementation could not be reused

`iteration.rs`: A single iteration of the interior-point method

`interiorpointmethod.rs`: The solver. The high-level management of the IPM algorithm.

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

ripopt: A Rust re-write from February 2026

Coded in 60/70 hs during the course of 6 weeks heavily relying on Claude Code [11].

According to the benchmarks CUTEst and xxxx, it already beats Ipopt.

Interesting insights into licensing issues and thought process documented in the repo.

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

Agenda

A tour of Ipopt
- History
- Techniques applied

An attempt of a simple IPM algorithm
- Challenges encountered
- Results

A tour of ripopt

Experiments
- What can we learn

Conclusion

Conclusion

Ipopt's performance can still be improved.

Different methods complement each other: By combining them, edge cases can be solved more efficiently.

Implementation of a state-of-the-art optimization package may be in the reach of modern AI agentic coding tools.

Questions?

Links

Presentation:
https://github.com/hlisdero/JMCS_Seminar_Applied_Optimization

Modified argmin:
https://github.com/hlisdero/argmin

ripopt:
https://github.com/jkitchin/ripopt

ipopt-rs:
https://github.com/elrnv/ipopt-rs

Bibliography

A. Wachter, *An interior point algorithm for large-scale nonlinear optimization with applications in process engineering*.
Carnegie Mellon University, 2002.

C.-O. Foundation, "Coin-op ipopt." https://github.com/coin-or/Ipopt.
Accessed: 2026-04-15.

R. H. Byrd, M. E. Hribar, and J. Nocedal, "An interior point algorithm for large-scale nonlinear programming," *SIAM Journal on Optimization*, vol. 9, no. 4, pp. 877–900, 1999.

R. J. Vanderbei, "Loqo: An interior point code for quadratic programming," *Optimization methods and software*, vol. 11, no. 1-4, pp. 451–484, 1999.

A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.

N. I. Gould, D. Orban, and P. L. Toint, "Cuter and sifdec: A constrained and unconstrained testing environment, revisited," *ACM Transactions on Mathematical Software (TOMS)*, vol. 29, no. 4, pp. 373–394, 2003.

COIN-OR Foundation, *Ipopt: Interior Point OPTimizer*, 2026.

S. Kroboth, "argmin: Mathematical optimization in pure rust." https://argmin-rs.org/.
Accessed: 2026-03-28.

S. Crozet, "nalgebra: Linear algebra library for rust." https://nalgebra.rs/.
Accessed: 2026-03-28.

S. Quiñones, "faer: linear algebra library for rust." https://faer.veganb.tw/.
Accessed: 2026-03-28.

J. Kitchin, "ripopt: A memory-safe interior point optimizer written in rust, inspired by ipopt." https://github.com/jkitchin/ripopt.
Accessed: 2026-04-11.